字符串

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 iii

第一章 字符串哈希 1
1.1 子串与反串双哈希 . . . 1

第二章 前缀函数与 KMP 算法 4
2.1 前缀函数 . . . 4
2.2 KMP 匹配 . . . 4

第三章 Z 函数（扩展 KMP） 6
3.1 Z 函数 . . . 6
3.2 两串扩展 KMP . . . 7

第四章 AC 自动机 8
4.1 AC 自动机出现次数 . . . 8

第五章 后缀数组 (SA) 11
5.1 后缀数组简介 . . . 11
5.1.1 线性后缀数组（SA-IS） . . . 11
5.1.2 倍增后缀数组与 LCP . . . 15
5.1.3 后缀 LCP 与子串比较 . . . 18
5.1.4 整数后缀数组的边界与 SAM 次数 . . . 19

第六章 后缀自动机 (SAM) 20
6.1 后缀自动机 . . . 20
6.2 SAM 字典序子串查询 . . . 23
6.3 SAM 最长公共子串 . . . 24

第七章 广义后缀自动机 26
7.1 广义后缀自动机：Trie BFS 构建 . . . 26
7.2 SAM 文档频率与阈值后缀 . . . 28
7.3 广义 SAM 点数与后缀语言 . . . 30

第八章 Manacher 31
8.1 Manacher 回文半径 . . . 31
8.2 回文半径与在线回文计数 . . . 32

第九章 回文树 33
9.1 回文树 PAM：回文后缀与出现次数 . . . 33

第十章 最小表示法 36
10.1 最小循环表示 . . . 36

第十一章 Lyndon 分解 38
11.1 Lyndon 分解：Duval . . . 38

第一部分 跨类依赖 **40**

附录 A 数论 **41**
A.1 模算术简介 41
A.1.1 静态模整数 41

附录 B 树状数组 **44**
B.1 树状数组第 k 小与组合依赖 44

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　字符串哈希

1.1　子串与反串双哈希

StringHash(s, base)，n<INT_MAX，两个底数分别在 [257,mod-2]，所有比较对象共享同组底数。get(l, r, rev=false) 取 0-based 半开区间；rev=true 反转原区间，空区间返回 {0,0}。join(a, b, len_b) 拼接，要求规范余数、同底数及 0≤len_b≤n，可拼出长度超过 n 的哈希。构造 O(n)，查询/拼接 O(1)，约 24(n+1) 字节；按 unsigned char 加 1 编码。判等必须先比长度；双哈希仍可能碰撞。理想独立均匀底数下，固定同长不同串的碰撞上界为两个 min(1,(L-1)/(mod[j]-258)) 之积，多次比较用联合界；可预测种子和适应性输入不享有该保证。需确定性时用直接比较或 SuffixArray + SuffixLCP；回文可用 Manacher。成员只读。

```
struct StringHash
{
    using H = array<int, 2>;
    static constexpr H mod = {1000000007, 1000000009};
    int n;
    H base;
    vector<H> pw, pre, rev;

    StringHash(const string &s, H b)
        : n(s.size()), base(b), pw(n + 1), pre(n + 1), rev(n + 1)
    {
        pw[0] = {1, 1};
        for (int j = 0; j < 2; j++)
        {
            assert(257 <= base[j] && base[j] <= mod[j] - 2);
            for (int i = 0; i < n; i++)
            {
                int x = (unsigned char)s[i] + 1;
                int y = (unsigned char)s[n - 1 - i] + 1;
                pw[i + 1][j] = 1LL * pw[i][j] * base[j] % mod[j];
                pre[i + 1][j] = (1LL * pre[i][j] * base[j] + x) % mod[j];
                rev[i + 1][j] = (1LL * rev[i][j] * base[j] + y) % mod[j];
            }
        }
    }
```

子串、反串与拼接（接上页同一结构体）：

```
    H get(int l, int r, bool reversed = false) const
    {
        assert(0 <= l && l <= r && r <= n);
        if (reversed)
        {
            int x = n - r;
            r = n - l;
            l = x;
        }
        const auto &h = reversed ? rev : pre;
        H ans;
        for (int j = 0; j < 2; j++)
        {
            long long x = h[r][j] - 1LL * h[l][j] * pw[r - l][j] % mod[j];
            ans[j] = (x + mod[j]) % mod[j];
        }
        return ans;
    }

    H join(H a, H b, int len_b) const
    {
        assert(0 <= len_b && len_b <= n);
        for (int j = 0; j < 2; j++)
        {
            a[j] = (1LL * a[j] * pw[len_b][j] + b[j]) % mod[j];
        }
        return a;
    }
};
```

模板题使用：P3370 【模板】字符串哈希

题意：输入 N 个非空字母数字串，输出不同串数。随机底数全局共享，按长度与双哈希去重；概率判等，AC 不证明零碰撞。

所需模板：StringHash（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3370

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    mt19937 rng(random_device{}());
    StringHash::H base;
    for (int j = 0; j < 2; j++)
    {
        base[j] = uniform_int_distribution<int>(257, StringHash::mod[j] - 2)(rng);
    }
    int n;
    cin >> n;
    set<pair<int, StringHash::H>> seen;
    while (n--)
    {
        string s;
        cin >> s;
        StringHash h(s, base);
        seen.insert({int(s.size()), h.get(0, s.size())});
    }
    cout << seen.size() << '\n';
}
```

第二章　前缀函数与 KMP 算法

2.1　前缀函数

prefix_function(s) 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
vector<int> prefix_function(const string &s)
{
    int n = (int)s.size();
    vector<int> p(n);
    for (int i = 1; i < n; i++)
    {
        int j = p[i - 1];
        while (j && s[i] != s[j]) j = p[j - 1];
        p[i] = j + (s[i] == s[j]);
    }
    return p;
}
```

联合使用示例见第 5 页（Luogu P3375），其中直接调用本模板。

2.2　KMP 匹配

代码依赖：prefix_function（第 4 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

kmp_match(s,t) 返回非空模式串 t 在 s 中的所有起点（0-based），允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
vector<int> kmp_match(const string &s, const string &t)
{
    assert(!t.empty());
    auto p = prefix_function(t);
    vector<int> ans;
    for (int i = 0, j = 0; i < (int)s.size(); i++)
    {
        while (j && s[i] != t[j]) j = p[j - 1];
        j += (s[i] == t[j]);
        if (j == (int)t.size())
        {
            ans.push_back(i - j + 1);
            j = p[j - 1];
        }
    }
    return ans;
}
```

模板题使用：Luogu P3375

题意：求模式串 t 在文本 s 中所有出现的位置，包含重叠匹配；模板返回 0-based 位置，输出加 1。最后一行直接输出 prefix_function(t)，它是模式串各前缀的最长真 border 长度。题目输入为非空大写串，函数按字符相等比较，不限于小写。

所需模板：prefix_function（第 4 页，2.1 节）、kmp_match（第 4 页，2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3375

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    cin >> s >> t;
    for (int p : kmp_match(s, t)) cout << p + 1 << '\n';
    for (int x : prefix_function(t)) cout << x << ' ';
    cout << '\n';
}
```

第三章　Z 函数（扩展 KMP）

3.1　Z 函数

z_function(s) 返回后缀 s[i..] 与原串的 LCP，约定 z[0]=n，空串返回空数组。O(n)。

```
vector<int> z_function(const string &s)
{
    int n = (int)s.size();
    vector<int> a(n);
    if (n) a[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++)
    {
        if (i < r) a[i] = min(r - i, a[i - l]);
        while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
        if (i + a[i] > r)
        {
            l = i;
            r = i + a[i];
        }
    }
    return a;
}
```

模板题使用：Library Checker zalgorithm

题意：给非空字符串，对每个起点求其后缀与原串的最长公共前缀长度。直接输出 Z 数组，z[0] 等于串长。

所需模板：z_function（第 6 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/zalgorithm

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    auto z = z_function(s);
    for (int i = 0; i < (int)z.size(); i++)
        cout << z[i] << (i + 1 == (int)z.size() ? '\n' : ' ');
}
```

3.2 两串扩展 KMP

代码依赖：z_function（第 6 页，3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

exkmp(s,t) 返回 {z,p}：z 为模式 t 的 Z 数组，z[0]=|t|；p[i] 是文本后缀 s[i..] 与 t 的 LCP 长度，均 0-based。空串合法，t 空时 p 全 0，s 空时 p 为空。任意字节相等比较，含零字节与高位字节；不拼接分隔符、不拷贝输入。需要 z_function。两串长度均须 <INT_MAX；O(|s|+|t|) 时间，两个输出 vector 共 O(|s|+|t|) 空间，返回时移动缓冲区。维护右端最远的匹配区间 [l,r)，区间内利用模式自身 Z 值再向右扩展。P5410 的 1-based 权值乘法应先提升到 long long。

```
// Return {Z(t), LCP(s[i..], t) for every i}; empty strings are allowed.
pair<vector<int>, vector<int>> exkmp(const string &s, const string &t)
{
    int n = s.size(), m = t.size();
    auto z = z_function(t);
    vector<int> p(n);
    for (int i = 0, l = 0, r = 0; i < n; i++)
    {
        if (i < r) p[i] = min(r - i, z[i - l]);
        while (p[i] < m && p[i] < n - i && s[i + p[i]] == t[p[i]])
            p[i]++;
        if (i + p[i] > r)
        {
            l = i;
            r = i + p[i];
        }
    }
    return {move(z), move(p)};
}
```

模板题使用：Luogu P5410 / 扩展 KMP

题意：输入小写串 a、b，长度均 1..20000000。求 b 的 Z 数组与 b 对 a 各后缀的 LCP 数组，各输出 xor((i+1)*(长度 +1))。z[0]=|b|；乘法先提升 long long。直接求两串 LCP，不拼接整串。

所需模板：z_function（第 6 页，3.1 节）、exkmp（第 7 页，3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5410

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string a, b;
    cin >> a >> b;
    auto [z, p] = exkmp(a, b);
    long long x = 0, y = 0;
    for (int i = 0; i < (int)z.size(); i++)
        x ^= 1LL * (i + 1) * (z[i] + 1);
    for (int i = 0; i < (int)p.size(); i++)
        y ^= 1LL * (i + 1) * (p[i] + 1);
    cout << x << '\n' << y << '\n';
}
```

第四章　AC 自动机

4.1　AC 自动机出现次数

add(s)/count(s) 默认小写 string；add(s,offset)/count(s,offset) 接受 string 或 vector<int>，每个元素减 offset 须在 0..25，同一自动机使用一致编码。字符串字面量请先转 string 再传双参数入口。add 返回终态，支持空模式返回根 0；count 返回逐状态重叠出现次数，根恒为文本长度 +1。全部插入后 build 一次，随后只读且可重复查询；重复模式共享状态。a[u].len 为 Trie 深度，go/fail 与 order 可读，order 不含根。size 用 a.size()，重置赋新对象。构建 O(26V+L)，查询 O(|text|+V)，空间 O(26V)。状态数、深度须在 int 内。非空旧用法不变；旧根计数语义需减 1。出现过的模式编号数仍按每个终点分别判断 cnt>0。

```
struct AhoCorasick
{
    struct Node
    {
        array<int, 26> go{};
        int fail = 0, len = 0;
    };

    vector<Node> a{Node{}};
    vector<int> order;
    bool built = false;

    template <class S>
    int add(const S &s, int offset)
    {
        assert(!built);
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            if (!a[u].go[c])
            {
                int v = (int)a.size();
                a[u].go[c] = v;
                a.push_back(Node{});
                a[v].len = a[u].len + 1;
            }
            u = a[u].go[c];
        }
        return u;
    }

    int add(const string &s)
    {
        return add(s, 'a');
    }
```

构建失败链接与统计出现次数（接上页同一结构体）：

```
    void build()
    {
        assert(!built);
        built = true;
        queue<int> q;
        for (int v : a[0].go)
            if (v) q.push(v);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            order.push_back(u);
            for (int c = 0; c < 26; c++)
            {
                int v = a[u].go[c];
                if (v)
                {
                    a[v].fail = a[a[u].fail].go[c];
                    q.push(v);
                }
                else
                    a[u].go[c] = a[a[u].fail].go[c];
            }
        }
    }

    template <class S>
    vector<long long> count(const S &s, int offset) const
    {
        assert(built);
        vector<long long> ans(a.size());
        ans[0] = 1;
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            u = a[u].go[c];
            ++ans[u];
        }
        for (int i = (int)order.size() - 1; i >= 0; i--)
            ans[a[order[i]].fail] += ans[order[i]];
        return ans;
    }

    vector<long long> count(const string &s) const
    {
        return count(s, 'a');
    }
};
```

模板题使用：Luogu P5357

题意：给多个小写字母模式和文本，输出每个模式的出现次数，允许重叠。先保存 add 返回的终止状态，全部插入后 build，再 count 文本并按状态取值；重复模式共享状态，仍按输入顺序输出。

所需模板：AhoCorasick（第 8 页，4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5357

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<int> terminal(n);
    for (int &id : terminal)
    {
        string pattern;
        cin >> pattern;
        id = ac.add(pattern);
    }
    ac.build();
    string text;
    cin >> text;
    auto count = ac.count(text);
    for (int id : terminal) cout << count[id] << '\n';
}
```

第五章　后缀数组 (SA)

5.1　后缀数组简介

5.1.1　线性后缀数组（SA-IS）

SAIS(s,alphabet)：整数符号 0..alphabet-1，1≤alphabet<INT_MAX，长度 n<INT_MAX；SAIS(string) 按 unsigned byte 处理，支持零字节。sa/rk/lcp 均长 n，0-based；lcp[0]=0，其余为相邻后缀 LCP。无需哨兵，空串结果为空。诱导排序时间/空间 O(n+alphabet)，递归深度 O(log n)；任意值离散化另需 O(n log n)。保留倍增 SuffixArray；SuffixLCP 不直接接受本类型。

```
struct SAIS
{
    vector<int> sa, rk, lcp;

    SAIS(const string &s)
    {
        assert(s.size() < INT_MAX);
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SAIS(a, 256);
    }

    SAIS(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX && 1 <= alphabet && alphabet < INT_MAX);
        for (int c : s) assert(0 <= c && c < alphabet);
        int n = s.size();
        sa = sort_suffixes(s, alphabet);
        rk.resize(n);
        lcp.assign(n, 0);
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) k++;
            lcp[rk[i]] = k;
            if (k) k--;
        }
    }
```

类型与桶内诱导（接上页同一结构体）：

```
private:
    static vector<int> sort_suffixes(const vector<int> &s, int alphabet)
    {
        int n = s.size();
        if (!n) return {};
        if (n == 1) return {0};
        vector<int> sa(n), lo(alphabet + 1), hi(alphabet + 1);
        vector<bool> type(n);
        for (int i = n - 2; i >= 0; i--)
            type[i] = s[i] == s[i + 1] ? type[i + 1] : s[i] < s[i + 1];
        for (int i = 0; i < n; i++)
            if (type[i]) lo[s[i] + 1]++;
            else hi[s[i]]++;
        for (int c = 0; c < alphabet; c++)
        {
            hi[c] += lo[c];
            lo[c + 1] += hi[c];
        }
        auto induce = [&](const vector<int> &lms)
        {
            fill(sa.begin(), sa.end(), -1);
            vector<int> pos = hi;
            for (int i : lms) sa[pos[s[i]]++] = i;
            pos = lo;
            sa[pos[s[n - 1]]++] = n - 1;
            for (int i = 0; i < n; i++)
            {
                int v = sa[i];
                if (v > 0 && !type[v - 1]) sa[pos[s[v - 1]]++] = v - 1;
            }
            pos = lo;
            for (int i = n - 1; i >= 0; i--)
            {
                int v = sa[i];
                if (v > 0 && type[v - 1]) sa[--pos[s[v - 1] + 1]] = v - 1;
            }
        };
```

LMS 命名与递归（接上页同一结构体）：

```
        vector<int> id(n, -1), lms;
        for (int i = 1; i < n; i++)
            if (!type[i - 1] && type[i])
            {
                id[i] = lms.size();
                lms.push_back(i);
            }
        induce(lms);
        int m = lms.size();
        if (m)
        {
            vector<int> order, reduced(m);
            for (int v : sa)
                if (id[v] >= 0) order.push_back(v);
            int names = 1;
            for (int i = 1; i < m; i++)
            {
                int a = order[i - 1], b = order[i];
                int x = id[a] + 1 < m ? lms[id[a] + 1] : n;
                int y = id[b] + 1 < m ? lms[id[b] + 1] : n;
                bool same = x - a == y - b;
                if (same)
                {
                    while (a < x && s[a] == s[b]) a++, b++;
                    same = a < n && b < n && s[a] == s[b];
                }
                if (!same) names++;
                reduced[id[order[i]]] = names - 1;
            }
            order = sort_suffixes(reduced, names);
            for (int &i : order) i = lms[i];
            induce(order);
        }
        return sa;
    }
};
```

模板题使用：Library Checker suffixarray

题意：输入长度 1..500000 的小写字符串，按字典序输出所有非空后缀的 0-based 起点。独立 SA-IS 实现；无需附加哨兵，不继承倍增版本的线上记录。

所需模板：SAIS（第 11 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/suffixarray

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SAIS suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        cout << suffix.sa[i] << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

5.1.2 倍增后缀数组与 LCP

构造 SuffixArray(s) 支持任意字节串，按 unsigned char 的 0..255 数值比较，包含内嵌零字节；也可构造 SuffixArray(a,alphabet)，整数序列要求 0≤a[i]<alphabet、1≤alphabet<INT_MAX。重新赋值 SuffixArray(a,alphabet) 可重建。长度须小于 INT_MAX。sa[k] 为第 k 个非空后缀起点，rk[i] 为排名，全部 0-based；lcp[k] 为 sa[k-1] 与 sa[k] 的最长公共前缀，lcp[0]=0。空序列合法，三数组均为空。无需手动追加哨兵，也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序，倍增 O(n log n+alphabet)，空间 O(n+alphabet)；字节串 alphabet=256。大整数先自行保序离散化，避免按最大原值开桶。Kasai 求 lcp 为 O(n)。不同起点 x,y 的 LCP 为 lcp[min(rk[x],rk[y])+1..max(rk[x],rk[y])] 的最小值；本结构不内建 RMQ，需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 n-x。覆盖倍增构造，不替代 DC3/SA-IS。

```
struct SuffixArray
{
    vector<int> sa, rk, lcp;

    SuffixArray(const string &s)
    {
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SuffixArray(a, 256);
    }

    SuffixArray(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX);
        assert(1 <= alphabet && alphabet < INT_MAX);
        int n = (int)s.size();
        sa.resize(n);
        rk.resize(n);
        lcp.assign(n, 0);
        if (!n) return;
        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
        int m = alphabet;
        for (int i = 0; i < n; i++)
        {
            assert(0 <= s[i] && s[i] < alphabet);
            rk[i] = s[i] + 1;
            cnt[rk[i]]++;
        }
        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;
```

```
        for (int k = 1; k < n; k *= 2)
        {
            int p = 0;
            for (int i = n - k; i < n; i++) order[p++] = i;
            for (int i : sa)
                if (i >= k) order[p++] = i - k;
            fill(cnt.begin(), cnt.begin() + m + 1, 0);
            for (int x : rk) cnt[x]++;
            for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
            old.swap(rk);
            auto second = [&](int i)
            {
                return i < n - k ? old[i + k] : 0;
            };
            m = 1;
            rk[sa[0]] = m;
            for (int i = 1; i < n; i++)
            {
                int a = sa[i - 1], b = sa[i];
                if (old[a] != old[b] || second(a) != second(b)) m++;
                rk[b] = m;
            }
            if (m == n) break;
        }
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
            lcp[rk[i]] = k;
            if (k) --k;
        }
    }
};
```

模板题使用：Luogu P3809

题意：把所有非空后缀按字典序排序，输出各后缀在原串中的起点。模板 sa 为 0-based，题目输出为 1-based，因此加 1。

所需模板：SuffixArray（第 15 页，5.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3809

```cpp
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    SuffixArray suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        std::cout << suffix.sa[i] + 1 << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

5.1.3 后缀 LCP 与子串比较

代码依赖：SuffixArray（第 15 页，5.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先构造 SuffixArray，再构造 SuffixLCP(sa)。重新赋值 SuffixLCP(sa) 可建新表。复制所需排名与 LCP 数据，不保存原字符串或源对象引用，源对象可销毁或重建。query(x,y) 返回 s[x..n) 与 s[y..n) 的最长公共前缀，0≤x,y≤n，允许空后缀；同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串，允许空串，返回 -1/0/1。若公共前缀覆盖较短子串，按长度比较，否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表 O(n log n) 时间、空间，每次查询 O(1)，无需哈希和原串。n、rk、lg、st 可读，禁止调用方破坏。前缀的最长公共后缀：在反串上构造 SuffixLCP rev，s[0..x) 与 s[0..y) 的最长公共后缀为 rev.query(n-x,n-y)。重复子串（AA 型）计数：枚举半长 len 与锚点 i,i+len，用正反两个 SuffixLCP 求向右、向左可延伸长度，再按区间差分累加。

```
struct SuffixLCP
{
    int n;
    vector<int> rk, lg;
    vector<vector<int>> st;

    SuffixLCP(const SuffixArray &suffix)
    {
        n = suffix.sa.size();
        rk = suffix.rk;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, {});
        st[0] = suffix.lcp;
        for (int k = 1; k <= lg[n]; k++)
        {
            int len = 1 << k;
            st[k].resize(n - len + 1);
            for (int i = 0; i <= n - len; i++)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
        }
    }
```

```
    int query(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= n);
        if (x == y || x == n || y == n) return n - max(x, y);
        int a = rk[x], b = rk[y];
        if (a > b) swap(a, b);
        int k = lg[b - a];
        return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
    }

    // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
    int compare(int l1, int r1, int l2, int r2) const
    {
        assert(0 <= l1 && l1 <= r1 && r1 <= n);
        assert(0 <= l2 && l2 <= r2 && r2 <= n);
        int a = r1 - l1, b = r2 - l2;
        if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
        return rk[l1] < rk[l2] ? -1 : 1;
    }
};
```

5.1.4 整数后缀数组的边界与 SAM 次数

SuffixArray 整数入口要求 $0 \le a[i] < alphabet$，大整数先保序离散化。sa/rk/lcp 从零编号，lcp[k] 比较 sa[k-1] 与 sa[k]，首项为零。LCP 需遇分隔符停止时，令 remain[i] 为 i 到下一分隔符前的普通字符数：

$$height[k] = \min\{lcp[k], remain[sa[k-1]], remain[sa[k]]\}.$$

两串拼接求最长公共子串只需检查异来源的相邻后缀。

SAM 状态 u 代表长度 $(len[link[u]], len[u]]$ 内同一 endpos 的子串。occ 种子真实状态为一、克隆为零，按 len 倒序传播（不能按创建编号）。根计数为 n，非空子串只看非根状态；次数乘长度用 long long。

第六章　后缀自动机 (SAM)

6.1　后缀自动机

extend(c) 接收整数 0..25，每次追加一个小写字符；last 为整串状态，a[0] 为根，根 link=-1。a[u].len 为状态最大长度，非根代表 (len[link[u]],len[u]] 且具有同一 endpos 集合。occ 是原始种子：真实扩展为 1、克隆为 0，不能直接当出现次数。counts() 按长度倒序汇总后返回副本，不改结构，重复查询及查询后继续追加安全；根计数为 n，非空子串只看 u>0。distinct() 为不同非空子串数。重赋值 SuffixAutomaton() 清空；固定 26 字母表构建 O(n)，counts/distinct 为 O(n)，空间 O(26n)。如需前缀位置对应状态，在每次 extend 后保存 last；本结构不输出广义 SAM 或最小 DFA 状态数。

```
struct SuffixAutomaton
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
        long long occ = 0;
    };

    vector<Node> a{Node{}};
    int last = 0;
```

接上页同一结构体：

```
    void extend(int c)
    {
        assert(0 <= c && c < 26);
        int cur = (int)a.size();
        a.push_back(Node{});
        a[cur].len = a[last].len + 1;
        a[cur].occ = 1;
        int p = last;
        while (p != -1 && !a[p].go[c])
        {
            a[p].go[c] = cur;
            p = a[p].link;
        }
        if (p == -1)
            a[cur].link = 0;
        else
        {
            int q = a[p].go[c];
            if (a[p].len + 1 == a[q].len)
                a[cur].link = q;
            else
            {
                int clone = (int)a.size();
                Node copy = a[q];
                a.push_back(copy);
                a[clone].len = a[p].len + 1;
                a[clone].occ = 0;
                while (p != -1 && a[p].go[c] == q)
                {
                    a[p].go[c] = clone;
                    p = a[p].link;
                }
                a[q].link = a[cur].link = clone;
            }
        }
        last = cur;
    }
```

接上页同一结构体：

```
    vector<long long> counts() const
    {
        vector<int> bucket(a[last].len + 1), order(a.size());
        vector<long long> ans(a.size());
        for (auto &v : a) ++bucket[v.len];
        partial_sum(bucket.begin(), bucket.end(), bucket.begin());
        for (int i = 0; i < (int)a.size(); i++)
        {
            order[--bucket[a[i].len]] = i;
            ans[i] = a[i].occ;
        }
        for (int i = (int)order.size() - 1; i > 0; i--)
            ans[a[order[i]].link] += ans[order[i]];
        return ans;
    }

    long long distinct() const
    {
        long long ans = 0;
        for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：给小写字母串，求不同非空子串的个数。extend 接收 0..25 的整数，故字符减去 a；所有字符追加后调用 distinct，结果使用 long long。

所需模板：SuffixAutomaton（第 20 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    cout << sam.distinct() << '\n';
}
```

6.2 SAM 字典序子串查询

SAMLex(sam,w={}) 引用已建好的 SuffixAutomaton 或已 build 的 GeneralSAM；索引使用期间 sam 必须存活且保持不变。默认每个非根状态权重 1，得到不同子串；传单串 sam.counts() 按出现次数计数。自定义 w 长度为状态数、非负 long long，每条到达该状态的串有 w[u] 份，根权重强制 0。sum[u] 为本状态及后续路径权重和，超过 LLONG_MAX 截断，仍正确支持所有 1≤k≤LLONG_MAX。kth(k,alphabet) 返回 optional<string>，不足或 k≤0 为 nullopt；默认 a..z，alphabet 必须是 26 字母的排列。短串先于其扩展，支持重复查询与自定义字母序。构建 O(26V+L)，空间 O(V+L)，L 为最长串长；单次查询 O(26× 答案长度)，额外空间为答案。weight/sum 只读，不支持扩展 SAM 后沿用旧索引。

```
// Keep sam alive and unchanged while this index is used.
template <class S>
struct SAMLex
{
    const S &sam;
    vector<long long> weight, sum;

    SAMLex(const S &sam, vector<long long> w = {}) : sam(sam), weight(move(w))
    {
        int n = sam.a.size(), length = 0;
        if (weight.empty()) weight.assign(n, 1);
        assert((int)weight.size() == n);
        weight[0] = 0;
        sum = weight;
        for (auto &v : sam.a) length = max(length, v.len);
        vector<int> bucket(length + 1), order(n);
        for (auto &v : sam.a) bucket[v.len]++;
        for (int i = 1; i <= length; i++) bucket[i] += bucket[i - 1];
        for (int i = 0; i < n; i++) order[--bucket[sam.a[i].len]] = i;
        for (int i = n - 1; i >= 0; i--)
        {
            int u = order[i];
            for (int v : sam.a[u].go)
                if (v) sum[u] += min(sum[v], LLONG_MAX - sum[u]);
        }
    }
```

字典序查询（接上页同一结构体）：

```
    // alphabet must be a permutation of a..z; k is 1-based.
    optional<string> kth(long long k, const string &alphabet = "abcdefghijklmnopqrstuvwxyz") const
    {
        if (k <= 0 || k > sum[0]) return nullopt;
        int u = 0;
        string ans;
        while (k > weight[u])
        {
            k -= weight[u];
            for (char c : alphabet)
            {
                int v = sam.a[u].go[c - 'a'];
                if (!v) continue;
                if (k > sum[v]) k -= sum[v];
                else
                {
                    ans += c;
                    u = v;
                    break;
                }
            }
        }
        return ans;
    }
};
```

6.3 SAM 最长公共子串

sam_lcs(sam,t) 接收已有单串 SuffixAutomaton 或已 build 的 GeneralSAM 及小写文本 t，返回 t 中的 {0-based 起点, 长度}；多解取 t 中最早起点，无公共非空串返回 {0,0}。广义 SAM 时求 t 与任一已加入文档的最长公共子串，不是所有文档公共串。t 可空，长度 <INT_MAX；构建 SAM 的成本另计，查询 O(|t|)、额外空间 O(1)，输入与 SAM 不变。沿后缀链接回退并同步匹配长度，不返回源文档坐标；需实际串时用 t.substr(start,length)。

```
// Return {start, length} in t; ties choose the earliest start.
template <class S>
pair<int, int> sam_lcs(const S &sam, const string &t)
{
    int p = 0, length = 0;
    pair<int, int> ans{0, 0};
    for (int i = 0; i < (int)t.size(); i++)
    {
        int c = t[i] - 'a';
        assert(0 <= c && c < 26);
        while (p && !sam.a[p].go[c])
        {
            p = sam.a[p].link;
            length = sam.a[p].len;
        }
```

```
        if (sam.a[p].go[c])
        {
            p = sam.a[p].go[c];
            length++;
        }
        else length = 0;
        if (length > ans.second) ans = {i - length + 1, length};
    }
    return ans;
}
```

第七章　广义后缀自动机

7.1　广义后缀自动机：Trie BFS 构建

先 add(s) 登记若干小写字符串，再 build() 原地将 Trie 变为广义 SAM；build 后禁止继续 add，可重复 build，重新收集串需新建对象。空串与重复串允许，distinct() 返回所有输入串中不同非空子串的总数，不包含跨串拼接。根为 0，a[v].go[c] 为转移、0 表示无边，link 为后缀链接、len 为最长代表串长度；从根沿字符转移可判定子串是否存在。a.size() 为按文档 endpos 等价类构造的状态数，不应直接称为后缀并集语言的最小 DFA 点数；ab、b 是两者不同的例子。P6139 官方样例一要求 endpos 点数 10，后缀语言最小化仅 7 点；该题用 a.size()，勿再最小化。固定 26 字符，设累计输入长度 L、Trie 点数 K，登记总时间 O(L)，build 时间与空间 O(26K)，状态少于 2K；累计输入长度及 2K 须可由 int 表示。克隆只复制已处理转移，不能带入尚未 BFS 处理的 Trie 子边。此版不提供出现次数或各串所属标记。新增 add(p,c) 在 build 前按 Trie 边登记，0<=c<26，p 为已有 Trie 节点；返回子节点号，重复边复用。单次均摊 O(1)，避免把输入树展开成多条长串；add(s) 复用此入口。原 Trie 编号在 build 后仍代表原根路径，但禁止继续 add。

```
struct GeneralSAM
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
    };

    vector<Node> a{Node{}};
    bool ready = false;

    // Trie step before build; repeated edges return the same node.
    int add(int p, int c)
    {
        assert(!ready && 0 <= p && p < (int)a.size());
        assert(0 <= c && c < 26);
        if (!a[p].go[c])
        {
            int v = a.size();
            a.push_back(Node{});
            a[p].go[c] = v;
        }
        return a[p].go[c];
    }

    void add(const string &s)
    {
        assert(!ready);
        int p = 0;
        for (char ch : s) p = add(p, ch - 'a');
    }
```

BFS 构建（接上页同一结构体）：

```
    void build()
    {
        if (ready) return;
        a.reserve(2 * a.size());
        queue<pair<int, int>> todo;
        for (int c = 0; c < 26; c++)
            if (a[0].go[c]) todo.push({0, c});
        while (!todo.empty())
        {
            auto [parent, c] = todo.front();
            todo.pop();
            int cur = a[parent].go[c], p = a[parent].link;
            a[cur].len = a[parent].len + 1;
            while (p != -1 && !a[p].go[c])
            {
                a[p].go[c] = cur;
                p = a[p].link;
            }
            if (p == -1)
                a[cur].link = 0;
            else
            {
                int q = a[p].go[c];
                if (a[p].len + 1 == a[q].len)
                    a[cur].link = q;
                else
                {
                    int clone = a.size();
                    Node copy = a[q];
                    copy.len = a[p].len + 1;
                    for (int &v : copy.go)
                        if (!a[v].len) v = 0;
                    a.push_back(copy);
                    while (p != -1 && a[p].go[c] == q)
                    {
                        a[p].go[c] = clone;
                        p = a[p].link;
                    }
                    a[q].link = a[cur].link = clone;
                }
            }
            for (int ch = 0; ch < 26; ch++)
                if (int v = a[cur].go[ch]; v && !a[v].len) todo.push({cur, ch});
        }
        ready = true;
    }
```

不同子串计数（接上页同一结构体）：

```
    long long distinct() const
    {
        assert(ready);
        long long ans = 0;
        for (int v = 1; v < (int)a.size(); v++) ans += a[v].len - a[a[v].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：本题只有一个字符串，输出不同非空子串数。广义模板可在 build 前多次 add；本例验证其单串特例，多串行为另有独立对拍。若只处理单串，原 SuffixAutomaton 更合适。此题不验证广义 SAM 状态数或最小 DFA 状态数。

所需模板：GeneralSAM（第 26 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    GeneralSAM sam;
    sam.add(s);
    sam.build();
    cout << sam.distinct() << '\n';
}
```

7.2 SAM 文档频率与阈值后缀

代码依赖：Fenwick（第 44 页，B.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

SAMDocuments(sam,s) 接收已完成构建的 GeneralSAM 及同一批小写文档 s；也兼容按该文档集完成的 OnlineSAM 或单文档 SuffixAutomaton。允许空串和重复串，重复输入按不同文档计数。索引期间 sam 须存活且不变。cnt[u] 是该状态任一非空代表串出现的文档数，cnt[0]=s.size() 约定空串出现在每篇文档；不能把单串出现次数当文档数。lengths(k) 要求 k>=1，返回每个状态最长代表串中、出现于至少 k 篇文档的最长后缀长度，无非空解为 0；k 大于文档数全 0。查询任意非空模式串时，从根转移成功后读取 cnt，缺边返回 0。L 为文档总长，V 为状态数，D 为文档数：构造 O((L+V)log(L+2)+D)，空间 O(L+V+D)，每次 lengths 为 O(V)。递归后缀链接树深度不超过最长文档长度加 1；保留递归，不模拟开栈。L、D、V 须小于 INT_MAX，数组可分配；a/g/cnt 视为只读。

```
template <class S>
struct SAMDocuments
{
    const S &sam;
    vector<vector<int>> g;
    vector<int> cnt;
```

文档标记与子树去重（接上页同一结构体）：

```
    SAMDocuments(const S &sam, const vector<string> &s)
        : sam(sam), g(sam.a.size()), cnt(sam.a.size())
    {
        int n = sam.a.size(), size = 0;
        vector<vector<int>> tag(n);
        for (int u = 1; u < n; u++) g[sam.a[u].link].push_back(u);
        for (int id = 0; id < (int)s.size(); id++)
        {
            int p = 0, len = 0;
            for (char c : s[id])
            {
                assert('a' <= c && c <= 'z');
                p = sam.a[p].go[c - 'a'];
                len++;
                assert(p && sam.a[p].len == len);
                tag[p].push_back(id);
                size++;
            }
        }
        Fenwick<int> bit(size);
        vector<int> last(s.size());
        int timer = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            int l = timer + 1;
            for (int id : tag[u])
            {
                if (last[id]) bit.add(last[id], -1);
                last[id] = ++timer;
                bit.add(timer, 1);
            }
            for (int v : g[u]) self(self, v);
            cnt[u] = bit.query(l, timer);
        };
        dfs(dfs, 0);
        cnt[0] = s.size();
    }
```

阈值后缀长度（接上页同一结构体）：

```
    // Longest suffix length occurring in at least k documents, for each state.
    vector<int> lengths(int k) const
    {
        assert(k >= 1);
        vector<int> best(cnt.size());
        auto dfs = [&](auto &&self, int u) -> void
        {
            if (cnt[u] >= k) best[u] = sam.a[u].len;
            for (int v : g[u])
            {
                best[v] = best[u];
                self(self, v);
            }
        };
        dfs(dfs, 0);
        return best;
    }
};
```

7.3 广义 SAM 点数与后缀语言

GeneralSAM 的状态按（文档编号，结束位置）的 endpos 集合等价。先 add 全部小写串，再 build；根为零，distinct 为不同非空子串数，a.size() 含根。构造后不再 add。多串 endpos 自动机不是后缀并集的最小 DFA，点数按 endpos 口径输出 a.size()。

第八章　Manacher

8.1　Manacher 回文半径

manacher(s) 返回 odd/even，下标从 0 开始。odd[i] 包含中心，闭区间 [i-odd[i]+1,i+odd[i]-1]，长度 2*odd[i]-1；even[i] 的中心在 i 前，区间 [i-even[i],i+even[i]-1]，长度 2*even[i]，零半径为空。空串合法，支持任意字节（含零），按字节而非 Unicode 字素比较；int 索引要求 n<2^30。最长回文取上述长度最大值；全部 odd/even 以 long long 求和得到回文出现总数。O(n) 时间和空间，边界显式检查，无需选择哨兵字符。输入为 string、vector<int> 或 vector<long long> 对象，支持空序列和任意整数；泛型 S 需 size()、随机下标及 O(1) 相等比较。字符串字面量请显式写 string("abc")。输入不拷贝，输出移动返回。

```
// odd[i] includes the center; even[i] is centered before i.
template <class S>
pair<vector<int>, vector<int>> manacher(const S &s)
{
    int n = (int)s.size();
    vector<int> odd(n), even(n);
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        odd[i] = k--;
        if (i + k > r)
        {
            l = i - k;
            r = i + k;
        }
    }
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
        even[i] = k--;
        if (i + k > r)
        {
            l = i - k - 1;
            r = i + k;
        }
    }
    return {move(odd), move(even)};
}
```

模板题使用：Library Checker enumerate_palindromes

题意：给非空字符串，从左到右输出每个字符及相邻字符间隙处的最长回文长度，共 2n-1 项。odd 含中心，even[i] 的中心在 i 前面，因此分别转成 2*odd[i]-1 和 2*even[i]。

所需模板：manacher（第 31 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_palindromes

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
    auto [odd, even] = manacher(s);
    vector<int> answer(2 * n - 1);
    for (int i = 0; i < n; i++)
    {
        answer[2 * i] = 2 * odd[i] - 1;
        if (i) answer[2 * i - 1] = 2 * even[i];
    }
    for (int i = 0; i < (int)answer.size(); i++)
        cout << answer[i] << (i + 1 == (int)answer.size() ? '\n' : ' ');
}
```

8.2 回文半径与在线回文计数

Manacher 的 odd[i] 包含中心，最长奇回文为 $[i - odd[i] + 1, i + odd[i] - 1]$；even[i] 以 i 前间隙为中心，最长偶回文为 $[i - even[i], i + even[i] - 1]$。odd/even 之和是回文出现总次数（long long），不是不同回文串数。按字节比较，空串合法，int 索引要求 $n < 2^{30}$。

PAM 的 suffixes 等于 fail 节点的 suffixes 加一；hits 只计最长后缀命中，完整次数用 occurrences()。total 最多 $n(n+1)/2$，distinct 才是不同非空回文串数。PAM 固定小写字母。

第九章　回文树

9.1　回文树 PAM：回文后缀与出现次数

add(ch) 追加小写英文字母，返回最长回文后缀节点，亦保存于 last。节点 0 是长度 0 根，1 是长度 -1 根，实际回文节点从 2 开始；distinct() 为不同非空回文串数。t[u].len 为长度，fail 为最长真回文后缀链接，suffixes 为该节点的非空回文后缀数量；追加后的 t[last].suffixes 就是本位置结尾的回文串数。total 为当前全部回文子串的出现次数之和。end 为首次出现时的 0-based 右端点，可恢复原串子段；hits 仅统计作为最长后缀的次数，不能当出现次数。occurrences() 返回按节点编号的完整出现次数副本，根置零，O(n)；重复调用以及查询后继续追加均安全。重新赋值 PalindromicTree() 清空重建。固定 26 字符集下建树 O(n)，空间 O(26n)。

```
struct PalindromicTree
{
    struct Node
    {
        int next[26]{};
        int len = 0, fail = 0, suffixes = 0, end = -1;
        long long hits = 0;
    };

    vector<Node> t;
    vector<int> s;
    int last = 0;
    long long total = 0;

    PalindromicTree() : t(2), s(1, -1)
    {
        t[0].fail = 1;
        t[1].len = -1;
    }

    int get_fail(int u) const
    {
        int pos = s.size() - 1;
        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
        return u;
    }
```

接上页同一结构体：

```
    // Append a lowercase letter; return the longest palindromic suffix node.
    int add(char ch)
    {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        s.push_back(c);
        int u = get_fail(last);
        if (!t[u].next[c])
        {
            int v = t.size();
            t.push_back(Node{});
            t[v].len = t[u].len + 2;
            t[v].fail = t[get_fail(t[u].fail)].next[c];
            t[v].suffixes = t[t[v].fail].suffixes + 1;
            t[v].end = (int)s.size() - 2;
            t[u].next[c] = v;
        }
        last = t[u].next[c];
        t[last].hits++;
        total += t[last].suffixes;
        return last;
    }

    int distinct() const { return (int)t.size() - 2; }

    // Copy-and-propagate: repeated calls and later appends remain valid.
    vector<long long> occurrences() const
    {
        vector<long long> count(t.size());
        for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
        for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
        count[0] = 0;
        count[1] = 0;
        return count;
    }
};
```

模板题使用：Luogu P5496

题意：按上一次答案把当前字符循环偏移后追加，输出以当前位置结尾的回文串个数。add 返回最长回文后缀节点，读取其 suffixes。

所需模板：PalindromicTree（第 33 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5496

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    PalindromicTree pam;
    int answer = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        char ch = 'a' + (s[i] - 'a' + answer) % 26;
        int u = pam.add(ch);
        answer = pam.t[u].suffixes;
        std::cout << answer << (i + 1 == (int)s.size() ? '\n' : ' ');
    }
}
```

第十章　最小表示法

10.1　最小循环表示

minimum_rotation(s) 返回字典序最小循环表示的起点 (0-based)，空串返回 0；按无符号字节比较，与 string 字典序一致。O(n)。

```
int minimum_rotation(const string &s)
{
    int n = (int)s.size(), i = 0, j = 1, k = 0;
    if (!n) return 0;
    while (i < n && j < n && k < n)
    {
        unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b)
        {
            ++k;
            continue;
        }
        if (a > b)
        {
            i += k + 1;
            if (i == j) ++i;
        }
        else
        {
            j += k + 1;
            if (i == j) ++j;
        }
        k = 0;
    }
    return min(i, j);
}
```

模板题使用：Luogu P13270【模板】最小表示法

题意：给定长度 1..10^7 的小写字母字符串，输出所有循环位移中字典序最小的字符串。minimum_rotation 返回 0-based 起点，使用 rotate 原地重排后输出；O(n) 时间，算法本体 O(1) 额外空间。

所需模板：minimum_rotation（第 36 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13270

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string s;
    if (!(cin >> n >> s)) return 0;
    int p = minimum_rotation(s);
    rotate(s.begin(), s.begin() + p, s.end());
    cout << s << '\n';
}
```

第十一章　Lyndon 分解

11.1　Lyndon 分解：Duval

lyndon(s) 返回严格递增的因子右端点 ends；按 0-based 半开区间 [0,ends[0])、[ends[i-1],ends[i]) 取因子，数值恰好也是 1-based 闭区间右端点。各因子严格小于自己的所有非空真后缀，因子序列按字典序非递增，分解唯一。输入按 unsigned char 的 0..255 字节序比较，支持空串、零字节与高位字节；不是 Unicode 字符顺序。空串返回空 vector，s 不变，长度须 ≤INT_MAX。Duval 时间 O(n)，预留 n 个 int 的输出容量，空间 O(n)，除返回数组外只需 O(1) 工作空间；不复制子串。P6114 直接异或 ends，无需再加 1；全部相同字符会拆成 n 个单字符，不能合并相等因子。

```
vector<int> lyndon(const string &s)
{
    assert(s.size() <= INT_MAX);
    int n = s.size();
    vector<int> ends;
    ends.reserve(n);
    int i = 0;
    while (i < n)
    {
        int j = i + 1, k = i;
        while (j < n && (unsigned char)s[k] <= (unsigned char)s[j])
        {
            if (s[k] == s[j])
                k++;
            else
                k = i;
            j++;
        }
        int len = j - k;
        while (i <= k)
        {
            i += len;
            ends.push_back(i);
        }
    }
    return ends;
}
```

模板题使用：P6114 【模板】Lyndon 分解

题意：输入小写字符串，输出非递增 Lyndon 分解的全部 1-based 右端点异或和；n≤5000001。

所需模板：lyndon（第 38 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6114

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int ans = 0;
    for (int r : lyndon(s))
        ans ^= r;
    cout << ans << '\n';
}
```

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

附录 B　树状数组

B.1　树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 44 页，B.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

算法索引

AhoCorasick, 8

exkmp, 7

Fenwick, 44

GeneralSAM, 26

kmp_match, 4

lyndon, 38

manacher, 31
minimum_rotation, 36
ModInt, 41

PalindromicTree, 33
prefix_function, 4

SAIS, 11
sam_lcs, 24
SAMDocuments, 28
SAMLex, 23
StringHash, 1
SuffixArray, 15
SuffixAutomaton, 20
SuffixLCP, 18

z_function, 6